



МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)

**Кафедра инструментального и прикладного программного обеспечения
(ИиППО)**

КУРСОВАЯ РАБОТА

по дисциплине: Бэкенд-разработка

по профилю: Разработка программных продуктов и проектирование информационных систем

направления подготовки: 09.03.04 «Программная инженерия»

Тема: Серверная часть веб-приложения для управления образовательными курсами

Студент: Миркин Кириль Леонидович

Группа: ИКБО-10-23

Руководитель: Волков Михаил Юрьевич, ст. преподаватель

Москва 2026



Цель и задачи работы

- Цель работы – разработка серверной части LMS-системы на основе Clean Architecture.
- Провести анализ предметной области и существующих LMS-решений.
- Обосновать выбор архитектурного подхода и технологического стека.
- Разработать архитектуру серверной части и структуру базы данных.
- Реализовать REST API, механизмы безопасности и автоматической проверки ответов.
- Показать взаимодействие серверной и клиентской частей системы.



Предметная область и требования

Предметная область

- учебные классы, уроки и теоретические материалы
- домашние задания и переиспользуемая база задач
- тестирование, оценивание и статистика
- взаимодействие преподавателя и студента через чат

Ключевые требования

- REST API для клиентского приложения
- Clean Architecture и слабая связанность слоёв
- PostgreSQL как основная СУБД
- межстраничная навигация и современный интерфейс



Анализ аналогов LMS-систем

Критерий	Moodle	Google Classroom	Stepik	Фоксфорд
Сильная сторона	гибкость и зрелое тестирование	простота запуска	мощная автопроверка	близость к школьной модели
Ограничение	сложность сопровождения	нет автопроверки и собственного сервера	нет модели закрытого класса	нет открытого API
Вывод для проекта	частичная база задач	код класса	автоматическая проверка	уроки, ДЗ, чат



Выбор архитектурного подхода

- DDD обеспечивает глубокое моделирование предметной области, но избыточен для текущего объёма системы.
- MVC/MVT удобен на старте, однако затрудняет изоляцию бизнес-логики от фреймворка.
- Clean Architecture обеспечивает явное разделение слоёв, независимость бизнес-правил и высокую тестируемость.



Технологический стек

- Python 3.12+ – актуальная версия языка с высокой производительностью и улучшенной поддержкой типизации.
- FastAPI – современный асинхронный фреймворк с Dependency Injection и OpenAPI.
- PostgreSQL – надёжная ACID-СУБД для прикладных систем среднего масштаба.
- SQLAlchemy 2.0 + Alembic – ORM и миграции, не зависящие от конкретного веб-фреймворка.
- React + TypeScript – клиентская часть, потребляющая REST API.



Web frameworks and technologies

The +5 point increase for FastAPI is one of the most significant shifts in the web framework space. This signals a strong trend towards using Python for building performant APIs and reflects the overall strength of the Python ecosystem.

? Which **web frameworks and web technologies** have you done extensive development work in over the past year, and which do you want to work in over the next year? (If you both worked with the framework and want to continue to do so, please check both boxes in that row.)

All Respondents

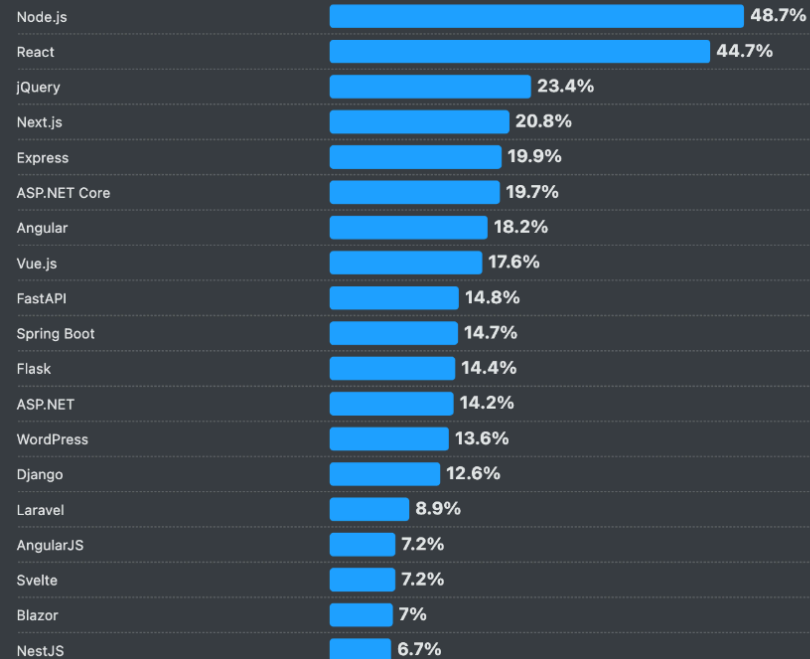
Professional Developers

Learning to Code

Professionals that Use AI

Learners that Use AI

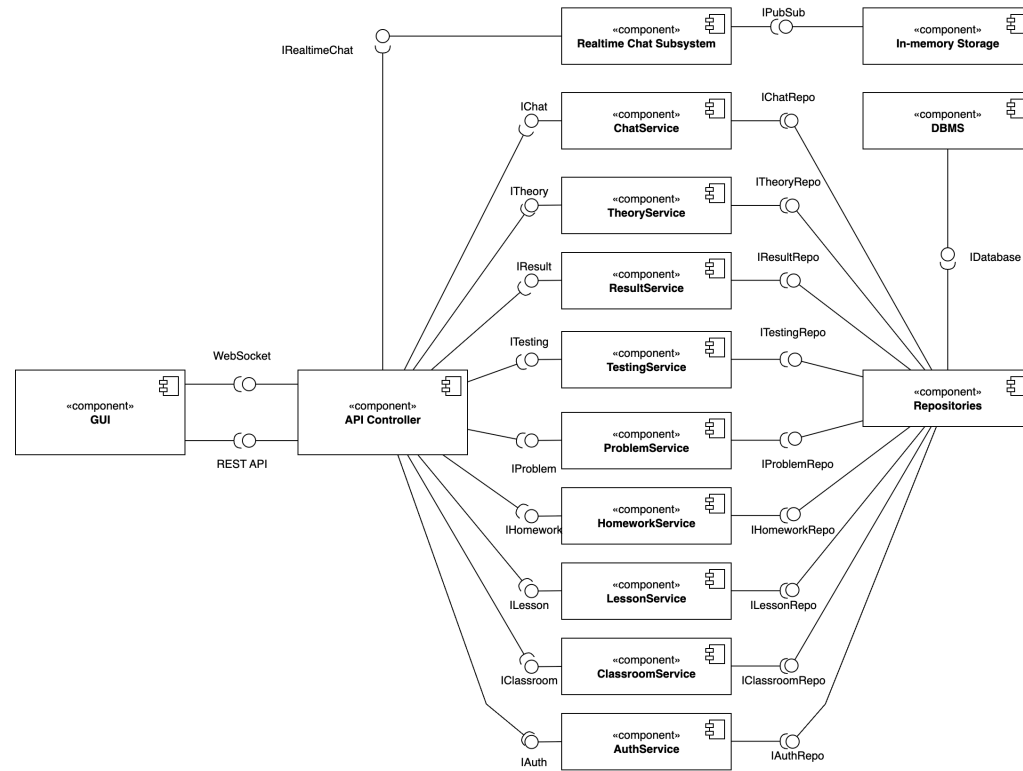
Write-Ins





Архитектура серверной части

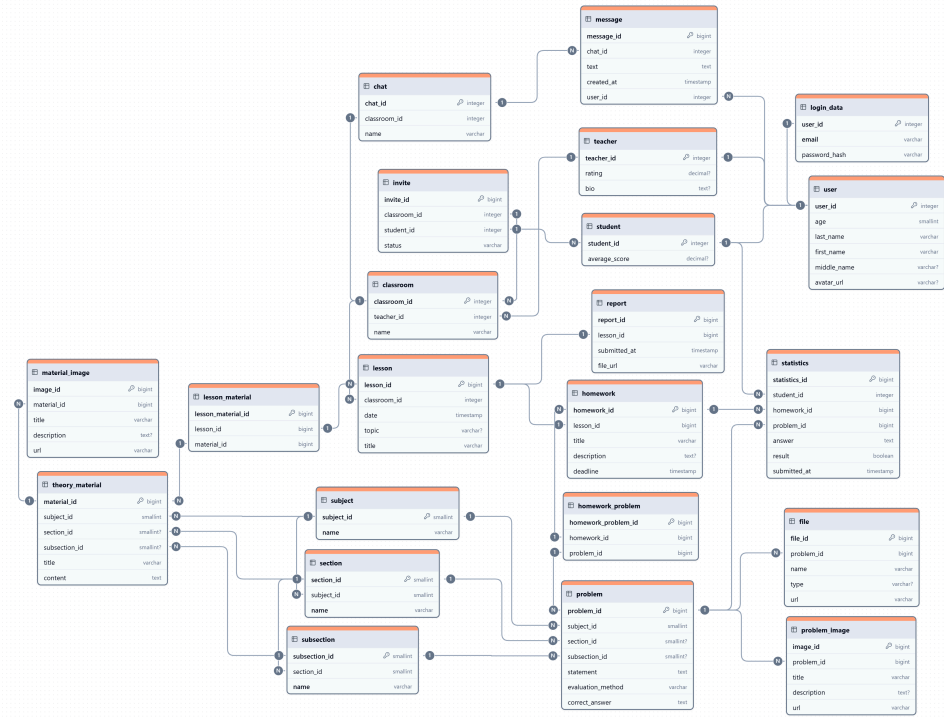
Система организована по принципам Clean Architecture: HTTP-слой делегирует выполнение сервисам, сервисы обращаются к репозиториям, а доступ к данным инкапсулирован на уровне ORM-моделей и PostgreSQL.





Структура базы данных

Ключевые сущности: users, login_data, classrooms, lessons, homework, problems, statistics, chat и message. Схема поддерживает разделение ролей, переиспользуемую базу задач и хранение истории взаимодействия.





REST API и OpenAPI-контракт

- группы эндпоинтов: auth, classrooms, lessons, homework, problems, testing, statistics, theory
- единый префикс /api/v1
- автоматическая генерация OpenAPI-спецификации
- единый контракт для backend и frontend



The screenshot displays the OpenAPI documentation for the 'Web Education Platform API' (version 1.0.0, OAS 3.1). The interface is split into two panes.

Left Pane: Health Endpoint

- Endpoint:** GET `/api/v1/health` (Health Check)
- Description:** Health check endpoint. Returns: dict: Application health status.
- Parameters:** No parameters.
- Execute:** A button to execute the request.
- Responses:**
 - Code:** 200
 - Response body:**

```
{
  "status": "healthy",
  "app_name": "Web Education Platform API",
  "version": "1.0.0",
  "timestamp": "2026-05-11T20:56:00.442684+00:00"
}
```

Right Pane: Endpoint List

- Classrooms:**
 - POST `/api/v1/classrooms` Create Classroom
 - GET `/api/v1/classrooms` Get My Classrooms
 - GET `/api/v1/classrooms/{classroom_id}` Get Classroom
 - PATCH `/api/v1/classrooms/{classroom_id}` Update Classroom
 - POST `/api/v1/classrooms/join` Join Classroom
 - GET `/api/v1/classrooms/{classroom_id}/students` Get Classroom Students
 - GET `/api/v1/classrooms/{classroom_id}/chat/messages` List Chat Messages
 - POST `/api/v1/classrooms/{classroom_id}/chat/messages` Post Chat Message
- Lessons:**
 - POST `/api/v1/lessons` Create Lesson
 - GET `/api/v1/lessons/classroom/{classroom_id}` Get Classroom Lessons
 - GET `/api/v1/lessons/{lesson_id}` Get Lesson
 - PATCH `/api/v1/lessons/{lesson_id}` Update Lesson
 - DELETE `/api/v1/lessons/{lesson_id}` Delete Lesson
- Homework:**
 - POST `/api/v1/homework` Create Homework
 - GET `/api/v1/homework/Lesson/{lesson_id}` Get Lesson Homework
 - GET `/api/v1/homework/{homework_id}` Get Homework
 - PATCH `/api/v1/homework/{homework_id}` Update Homework
 - DELETE `/api/v1/homework/{homework_id}` Delete Homework
 - GET `/api/v1/homework/{homework_id}/problems` Get Homework Problems

OpenAPI-документация используется как машинно-читаемое описание интерфейса серверной части.



Безопасность серверной части

- Argon2id используется для хеширования паролей.
- JWT применяется для безсессионной аутентификации.
- роль пользователя проверяется при доступе к защищённым ресурсам.
- ошибки обработки запросов возвращаются в унифицированном формате.

```
_MEMORY_COST_KIB = 1024 * 64
ph = argon2.PasswordHasher(
    time_cost=2,
    memory_cost=_MEMORY_COST_KIB,
    parallelism=1,
)

def create_access_token(data: dict)
-> str:
    payload = {**data, "exp":
utc_now() + ttl}
    return jose_jwt.encode(payload,
SECRET_KEY, algorithm=ALGORITHM)
```



Ключевой сценарий: автоматическая проверка ответа

- проверка прав доступа студента
- проверка существования задания и задачи
- поиск задачи в составе домашнего задания
- сравнение ответа с эталоном
- обновление балла, числа попыток и времени

```
async def submit_answer(...):  
    stats = await  
self.stats_repo.get_or_create_stats(...)  
    is_correct =  
self._check_answer(answer,  
correct_answer)  
    new_score = min(stats.score +  
points, stats.max_score)  
    updated = await  
self.stats_repo.update(stats.id,  
{...})  
    return  
StatisticsResponse.model_validate(updated)
```



Тестирование серверной части

- pytest используется для автоматизированной проверки бизнес-ЛОГИКИ
- каждый тест выполняется в отдельной схеме PostgreSQL
- проверяются AuthService, ChatService, ProblemService, ResultService, TheoryService и realtime-модули
- изолированное тестирование подтверждает корректность Clean Architecture

```
> pytest tests/ -v --tb=short --color=yes -ra

===== test session starts =====
platform darwin -- Python 3.13.0, pytest-8.4.2, pluggy-1.6.0 -- /Users/kerrrodar/Dev/wep-education-platform/backend/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /Users/kerrrodar/Dev/wep-education-platform/backend
configfile: pyproject.toml
plugins: anyio-4.12.0
collected 30 items

tests/test_auth_service.py::test_register_user_teacher[asyncio] PASSED [ 3%]
tests/test_auth_service.py::test_register_user_student[asyncio] PASSED [ 6%]
tests/test_auth_service.py::test_switch_role_creates_profile_and_updates_active_role[asyncio] PASSED [ 10%]
tests/test_auth_service.py::test_old_token_becomes_invalid_after_role_switch[asyncio] PASSED [ 13%]
tests/test_auth_service.py::test_register_duplicate_email[asyncio] PASSED [ 16%]
tests/test_auth_service.py::test_authenticate_success[asyncio] PASSED [ 20%]
tests/test_auth_service.py::test_authenticate_wrong_password[asyncio] PASSED [ 23%]
tests/test_auth_service.py::test_authenticate_nonexistent_user[asyncio] PASSED [ 26%]
tests/test_chat_pagination.py::test_message_repo_tail_and_before_id[asyncio] PASSED [ 30%]
tests/test_chat_service.py::test_chat_service_teacher_can_post_and_list[asyncio] PASSED [ 33%]
tests/test_chat_service.py::test_chat_service_rejects_non_member_student[asyncio] PASSED [ 36%]
tests/test_openapi_contract.py::test_openapi_contains_error_response_schema PASSED [ 40%]
tests/test_openapi_contract.py::test_openapi_operations_have_standard_error_responses PASSED [ 43%]
tests/test_openapi_contract.py::test_openapi_progress_endpoints_are_typed PASSED [ 46%]
tests/test_problem_service.py::test_create_problem[asyncio] PASSED [ 50%]
tests/test_problem_service.py::test_get_all_problems[asyncio] PASSED [ 53%]
tests/test_problem_service.py::test_get_problem_by_id[asyncio] PASSED [ 56%]
tests/test_problem_service.py::test_get_nonexistent_problem[asyncio] PASSED [ 60%]
tests/test_problem_service.py::test_update_problem[asyncio] PASSED [ 63%]
tests/test_problem_service.py::test_delete_problem[asyncio] PASSED [ 66%]
tests/test_realtime_auth.py::test_extract_bearer_token PASSED [ 70%]
tests/test_realtime_auth.py::test_extract_websocket_token_prefers_query_param PASSED [ 73%]
tests/test_realtime_auth.py::test_extract_websocket_token_falls_back_to_header PASSED [ 76%]
tests/test_realtime_connection_manager.py::test_connection_manager_connect_broadcast_disconnect[asyncio] PASSED [ 80%]
tests/test_realtime_presence_store.py::test_presence_store_online_filters_stale_members[asyncio] PASSED [ 83%]
tests/test_realtime_presence_store.py::test_presence_store_typing_stops_on_offline[asyncio] PASSED [ 86%]
tests/test_result_service.py::test_result_service_classroom_progress_aggregates[asyncio] PASSED [ 90%]
tests/test_result_service.py::test_result_service_student_progress_aggregates[asyncio] PASSED [ 93%]
tests/test_theory_service.py::test_theory_service_filters_unpublished_for_students[asyncio] PASSED [ 96%]
tests/test_token_role_mismatch.py::test_get_current_user_rejects_role_mismatch[asyncio] PASSED [100%]
```



Клиентское представление и интеграция

Ученики класса

Всего учеников: 5

Ученик	Средний балл	Ожидают проверки	Последняя активность	
Иванов Иван ivanov.i@school.edu	85%	2	10 мая, 14:30	Просмотр работ
Петров Алексей petrov.a@school.edu	72%	1	9 мая, 16:45	Просмотр работ
Сидорова Мария sidorova.m@school.edu	95%	0	10 мая, 10:15	Просмотр работ
Козлов Дмитрий kozlov.d@school.edu	60%	1	8 мая, 12:20	Просмотр работ
Новикова Анна novikova.a@school.edu	78%	0	10 мая, 09:30	Просмотр работ

Экспорт списка

© 2025 WEP. Все права защищены.

Интерфейс преподавателя: работа с классом, учениками и кодом приглашения.

Открыть описание домашнего задания

Найдите корни уравнения: $x^2 - 4 = 0$ и заполните таблицу соответствия между пунктами.

		Номер пункта						
		1	2	3	4	5	6	7
Номер пункта	1	*	*	*	*			
	2	*				*		
	3	*	*			*		
	4	*	*		*	*	*	
	5			*	*	*	*	*
	6	*	*		*	*	*	*
	7				*	*	*	*

Ваш ответ

Вложения:
Нет вложений

Прикрепить файл

Сохранить ответ

© 2025 WEP. Все права защищены.

Интерфейс студента: выполнение домашнего задания и отправка ответа.

Frontend реализован на React + TypeScript по методологии Feature-Sliced Design и взаимодействует с сервером через OpenAPI-контракт и axios-клиент.



Основные результаты работы

- Проведён анализ предметной области и LMS-аналогов.
- Обоснован выбор Clean Architecture и технологического стека.
- Разработана архитектура серверной части и схема базы данных.
- Реализованы REST API, аутентификация, автопроверка, статистика и чат.
- Подготовлена клиентская часть как потребитель REST API.
- Корректность решений подтверждена автоматизированным тестированием.



Спасибо за внимание!
